

Adaptive Guardrail Layer (AGL): Hybrid Detection for Securing LLM Prompts

Real-Time Defense Against Prompt Injection, Jailbreaking, and Adversarial Inputs

Alexander J Padin

Master of Science in Applied
Artificial Intelligence

Shiley Marcos School of
Engineering / University of
San Diego

apadin@sandiego.edu

Joel Dievendorf

Master of Science in Applied
Artificial Intelligence

Shiley Marcos School of
Engineering / University of
San Diego

jdievendorf@sandiego.edu

Jack Baxter

Master of Science in Applied
Artificial Intelligence

Shiley Marcos School of
Engineering / University of
San Diego

jackbaxter@sandiego.edu

ABSTRACT

Large language models (LLMs) introduce a new class of security vulnerabilities in which adversarial inputs, such as prompt injection and jailbreak attacks, manipulate model behavior through natural language rather than code-level exploits. This project addresses this problem by developing the Adaptive Guardrail Layer (AGL), a prompt-level security system trained on a composite dataset of approximately 100,000 labeled examples aggregated from publicly available adversarial prompt corpora and supplemented with benign and synthetic data.

AGL is implemented as a hybrid detection pipeline that combines a fine-tuned RoBERTa Transformer classifier with a denoising autoencoder for anomaly detection. Incoming prompts are processed through both components, and their outputs are aggregated using a rule-based decision layer that produces a binary malicious-versus-benign classification. For malicious prompts, a downstream categorization

module assigns an attack subtype to provide additional threat context. The system is deployed as an end-to-end application with a FastAPI backend and web-based interface, enabling real-time prompt evaluation with a consistent feature-engineering pipeline across training and inference.

On a synthetic, balanced evaluation dataset of 2,000 prompts, AGL achieved an accuracy of 0.952, precision of 0.912, and recall of 1.000 for malicious prompt detection, with an average latency of approximately 102 milliseconds per request. These results demonstrate that combining supervised classification with anomaly detection improves detection coverage while maintaining performance suitable for real-time deployment, supporting the use of hybrid architectures for securing LLM-based systems.

KEYWORDS

Large Language Models (LLMs), Prompt Injection, Jailbreaking, Adversarial Inputs, LLM

Security, Guardrail Systems, Binary Classification, Anomaly Detection, Denoising Autoencoder, RoBERTa, Transformer Models, Out-of-Distribution Detection, Natural Language Processing (NLP), DevSecOps, AI Security

1 Introduction

As organizations increasingly integrate LLMs into customer-facing products and internal systems, a new category of security threat has emerged. Unlike traditional software exploits, which target code-level vulnerabilities such as SQL injection or buffer overflows, adversarial attacks against LLMs operate at the natural-language interface, manipulating model behavior through carefully crafted inputs rather than malicious code (Greshake et al., 2023). These attacks, including prompt injection, jailbreaking, and data exfiltration, are now recognized as critical risks by the Open Worldwide Application Security Project (OWASP, 2023). For enterprises, the consequences range from unauthorized disclosure of system instructions and generation of harmful content to regulatory non-compliance and financial loss.

This project introduces the AGL, a real-time prompt security system designed to detect and intercept adversarial inputs before they reach an underlying LLM. AGL operates as a first-stage defense layer deployed at the API boundary, analyzing each incoming prompt and performing binary classification to determine whether the input is malicious or benign. For prompts identified as malicious, the system applies a downstream categorization step to assign an attack subtype, such as prompt injection, jailbreak, or data exfiltration. This two-stage design separates detection from interpretation,

enabling reliable malicious prompt detection while still providing actionable threat context.

AGL is designed for DevSecOps teams, AI platform engineers, and organizations deploying LLM-powered systems at scale. In a production environment, it functions as middleware between users and LLM services, screening all incoming prompts in real time with minimal latency. The system is trained on a composite dataset aggregated from multiple publicly available adversarial prompt corpora, including the Lakera Mosaic Prompt Injection dataset (Lakera AI, 2024), the deepset Prompt Injections dataset (deepset, 2023), and jailbreak datasets compiled from in-the-wild attacks (Shen et al., 2023). These sources are supplemented with benign conversational prompts and synthetic adversarial examples to improve class balance and coverage. In a live deployment, the dataset could be continuously updated using flagged prompts reviewed by analysts, enabling the system to adapt to evolving attack patterns.

The core architecture of AGL combines a fine-tuned Transformer-based classifier, specifically RoBERTa (Liu et al., 2019), with an anomaly detection module designed to identify out-of-distribution inputs. This hybrid design addresses a key limitation of supervised classifiers: their dependence on labeled training data. While the classifier captures known adversarial patterns, the anomaly detector identifies inputs that deviate from the learned distribution of benign prompts, improving coverage against previously unseen or obfuscated attacks.

The central hypothesis of this work is that combining supervised classification with anomaly detection will improve detection

reliability, particularly recall, compared to single-model approaches, while maintaining latency suitable for real-time deployment. This design prioritizes minimizing false negatives, reflecting the security requirement that malicious prompts should not bypass the system.

This work makes four primary contributions. First, it proposes a hybrid prompt-screening architecture that integrates Transformer-based classification with anomaly detection to improve detection coverage across both in-distribution and out-of-distribution prompt patterns. Second, it introduces a rule-based decision layer that aggregates model outputs into an interpretable and controllable risk score for final classification. Third, it incorporates a post hoc attack categorization module that enriches detected malicious prompts with structured threat context. Finally, it implements AGL as a complete end-to-end system, including a reusable feature-engineering pipeline, a FastAPI-based inference service, and a web-based user interface for real-time prompt evaluation.

2 Data Summary

The AGL is trained and evaluated using a composite dataset of approximately 105,000 labeled prompt examples aggregated from multiple publicly available adversarial prompt corpora. Each data point consists of a raw natural-language user prompt and a corresponding security label indicating whether the input is malicious or benign. Prompts vary significantly in length and complexity, ranging from short conversational queries to multi-step adversarial instructions that incorporate role-playing, obfuscation, or encoded payloads. While some source datasets include multi-class labels (e.g., jailbreak, injection), all labels were

standardized into a unified binary schema to align with the system’s primary detection objective.

The dataset was constructed from several sources, including WildGuardMix (Allen AI, 2024), the Prompt Injection and Benign Prompt Dataset (CyberPrince, 2024), the Malicious Prompt Detection Dataset (Jebbar, 2024), Malicious LLM Prompts (Codesagar, 2024), the deepset Prompt Injections dataset (deepset, 2023), and the Jailbreak Classification dataset (Hao, 2023). These were supplemented with benign conversational prompts and synthetic adversarial examples to improve class balance and coverage of underrepresented attack patterns. After aggregation, labels were normalized and validated to ensure consistency across datasets.

Several data quality challenges were identified during preprocessing. The primary issue is the lack of standardized labeling across sources, as most publicly available datasets provide only binary classifications or inconsistent definitions of attack types. This limitation reinforces the decision to frame the task as binary malicious prompt detection, with attack categorization handled as a downstream enrichment step rather than a primary learning objective. In addition, adversarial prompts frequently contain obfuscation techniques such as special characters, structured formatting, or encoded text. Rather than removing these artifacts, the preprocessing pipeline normalizes formatting inconsistencies while preserving obfuscation as an informative signal for detection.

In addition to structural features, frequency-based text analysis highlights recurring linguistic patterns within malicious prompts. As illustrated in Figure 2.4, adversarial inputs frequently contain instruction-oriented language, imperative phrasing, and task-driven constructs that attempt to guide or override model behavior. While similar vocabulary may appear in benign prompts, the context and structure of usage differ, reinforcing the need for models capable of capturing semantic relationships rather than relying on simple keyword matching.

Relationships between variables also indicate moderate correlation among length-based features, such as word count and token count. While this introduces some redundancy in the raw feature space, it does not negatively impact performance when using deep learning architectures, which can learn nonlinear feature interactions. For anomaly detection, feature preprocessing includes normalization and dimensionality reduction to mitigate the effects of correlated inputs and improve model stability.

Overall, the dataset provides strong signal alignment with the project goal of malicious prompt detection. The combination of diverse adversarial sources, balanced class distribution, and observable structural and linguistic differences between classes supports the use of hybrid modeling approaches that integrate supervised classification with anomaly detection. These characteristics are critical for improving robustness against both known attack patterns and previously unseen, out-of-distribution inputs.

3 Literature Review

The increasing adoption of large language models (LLMs) in production systems has introduced a new class of vulnerabilities at the

natural-language interface layer. Unlike traditional software exploits, adversarial attacks such as prompt injection and jailbreaks manipulate model behavior through crafted inputs, enabling unauthorized actions, data leakage, or policy bypass (Greshake et al., 2023; OWASP, 2023). These attacks exploit the flexibility of natural language systems, making them difficult to detect using conventional security mechanisms.

Early mitigation strategies relied primarily on rule-based filtering approaches, including keyword blocklists and pattern matching. While computationally efficient, these methods are limited in their ability to detect obfuscated or novel attacks, as they depend on predefined patterns and lack contextual understanding (Shen et al., 2023). As adversarial techniques evolve to include paraphrasing, encoding, and multi-step instruction chaining, static filtering approaches become increasingly ineffective.

Recent research has shifted toward machine learning-based detection, particularly using Transformer architectures such as BERT (Devlin et al., 2019) and RoBERTa (Liu et al., 2019). These models leverage self-attention mechanisms to capture contextual relationships between tokens, enabling them to identify nuanced semantic and structural patterns in text. Prior work has demonstrated that fine-tuned Transformer-based classifiers outperform rule-based systems in detecting adversarial prompts, particularly in cases involving paraphrasing or indirect manipulation (Zhang et al., 2024). These findings highlight the importance of modeling both linguistic context and structural complexity in adversarial input detection.

In parallel, anomaly detection techniques have been explored to address the limitations of supervised learning. Supervised classifiers rely on labeled training data and therefore struggle to generalize to previously unseen attack patterns. Approaches such as deep one-class classification and autoencoder-based methods aim to model the distribution of benign inputs and identify deviations from this distribution as potential anomalies (Ruff et al., 2018). These methods are particularly useful for detecting out-of-distribution inputs that may not resemble known attack patterns but still pose security risks.

Several commercial and open-source guardrail systems have emerged to address LLM prompt security in operational environments. Lakera Guard provides real-time prompt screening as middleware, achieving high precision but relatively low recall, indicating that a significant portion of adversarial inputs may go undetected (Palit et al., 2025). Rebuff employs a multi-layered defense strategy combining heuristic and machine learning-based approaches, achieving improved recall at the cost of substantially higher latency and false positive rates, limiting its practicality for real-time deployment. Other open-source efforts, including models released by ProtectAI, further demonstrate the growing interest in scalable prompt security solutions.

Table 3.1. Comparison of Existing LLM Guardrail System Performance

System	Method	Precision	Recall	Latency (s/prompt)
Lakera Guard	Proprietary ML pipeline	0.964	0.501	0.066
Rebuff	Multi-layer heuristic + ML	—	Higher	29.33
ProtectAI	Open-source	—	—	—

v2	classifier			
AGL (proposed)	Fine-tuned RoBERTa + Anomaly Detection	0.912	1	0.102

Note: AGL metrics are based on synthetic API-based evaluation and are included for contextual comparison.

Table 3.1 summarizes the reported performance characteristics of representative guardrail systems. As shown, existing solutions exhibit trade-offs between precision, recall, and latency, with no single system achieving strong performance across all dimensions simultaneously.

The performance trade-offs observed in Table 3.1 highlight a key challenge in prompt security systems. High-precision approaches reduce false positives but may fail to detect more subtle or novel attacks, while high-recall systems often introduce latency or usability constraints. Additionally, many existing approaches rely on a single detection mechanism, limiting their ability to simultaneously capture both known attack patterns and previously unseen inputs.

Collectively, existing research demonstrates that Transformer-based models provide strong capabilities for adversarial prompt classification, while anomaly detection methods offer complementary strengths in identifying out-of-distribution inputs. However, no single approach fully addresses the combined challenges of detection coverage, generalization to novel attacks, and real-time performance requirements. This gap motivates the exploration of architectures that integrate multiple detection strategies to improve robustness in practical deployment settings.

4 Methodology

The AGL is implemented as a hybrid prompt-screening pipeline that combines supervised classification, unsupervised anomaly detection, and a rule-based decision layer. As illustrated in Figure 4.1, each input prompt is processed through both the classification and anomaly detection components, and their outputs are aggregated into a final binary decision, followed by an optional attack categorization stage for malicious inputs. This architecture enables detection of both known adversarial patterns and previously unseen inputs that fall outside the distribution of benign prompts, while maintaining interpretability in the final decision.

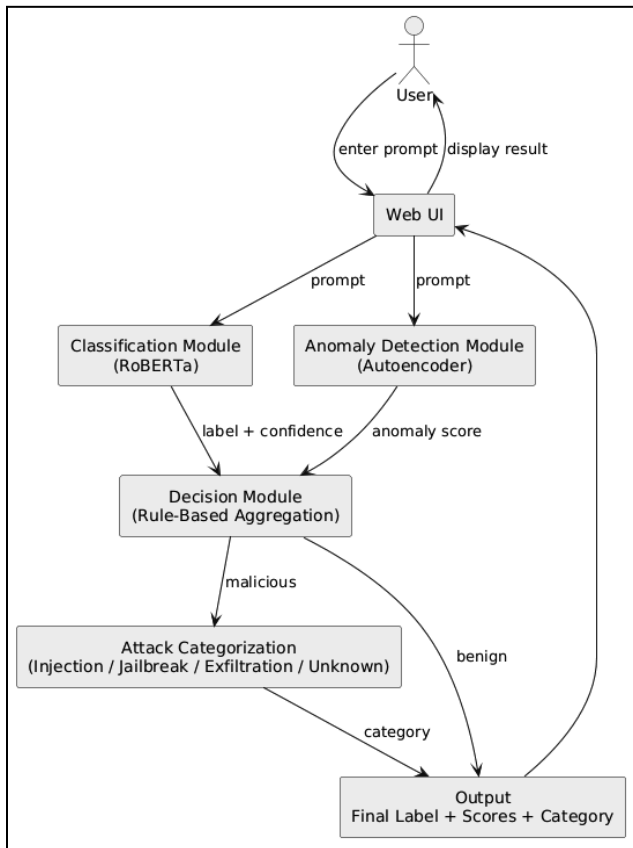


Figure 4.1. High-level architecture of the Adaptive Guardrail Layer (AGL) pipeline

4.1 Binary Classification

The primary classification component is a fine-tuned RoBERTa encoder adapted for sequence classification (Liu et al., 2019). Given a raw prompt, the model performs tokenization, padding, and truncation before producing a probability distribution over the binary classes via a softmax output layer. The predicted class is selected using an argmax operation, and the associated probability is retained as a confidence score. A minimum confidence threshold is applied to identify low-confidence predictions, which are treated as uncertain during downstream decision-making. RoBERTa was selected for its ability to capture contextual and semantic relationships in adversarial prompts, including multi-step instructions, roleplay framing, and indirect manipulation.

To complement the classifier, AGL incorporates an unsupervised anomaly detection module to identify inputs that deviate from expected benign behavior. The final anomaly detection model is a denoising deep autoencoder trained exclusively on benign prompts and implemented from scratch in PyTorch, without relying on pre-built model architectures. Rather than operating on raw text, the model uses a persisted feature-engineering pipeline that converts each prompt into a structured numerical representation. These features include lexical and structural indicators (e.g., token counts, punctuation ratios, whitespace patterns, non-ASCII characters, and formatting markers), phrase-based indicators for behaviors such as instruction override, roleplay, and obfuscation, and dense semantic embeddings generated by a sentence-transformer model and reduced via principal component analysis (PCA). The resulting feature vectors are normalized using saved preprocessing artifacts to ensure consistency between training and inference.

4.2 Anomaly Detection & Decision Aggregate

The autoencoder is implemented as a fully connected encoder-decoder network with nonlinear activation functions and a compressed latent representation. During training, noise is injected into the input features to improve robustness. At inference, the model reconstructs the input feature vector and computes the mean squared reconstruction error (MSE), which serves as the anomaly score. Prompts are flagged as anomalous when this score exceeds a calibrated threshold, indicating deviation from the learned benign distribution.

The outputs of the classifier and anomaly detector are combined through a rule-based decision module that aggregates both signals into a cumulative risk score. This module computes a weighted combination of anomaly magnitude and classification confidence. High-confidence malicious predictions contribute strong positive weight, while low-confidence predictions contribute smaller adjustments. Anomalous inputs increase the score, with additional weight applied when anomaly signals coincide with classifier uncertainty or malicious predictions. Confident benign predictions reduce the score. The final classification is determined by comparing the aggregated score against a predefined threshold, producing a binary malicious or benign label. This design provides an interpretable mechanism for resolving agreement, disagreement, and uncertainty between the two models.

For prompts classified as malicious, AGL applies a downstream attack categorization stage. This module assigns one of several categories, including prompt injection, jailbreak, exfiltration, or unknown malicious behavior. When trained

subtype models are available, categorization is performed using a TF-IDF vectorizer combined with a Gradient Boosting classifier. Otherwise, the system falls back to deterministic heuristic rules based on pattern matching. These rules detect indicators such as instruction override phrases, safety bypass attempts, and data exfiltration requests. The categorization output is returned alongside the classification result to provide structured threat context.

In addition to attack categorization, the system computes a severity score based on classifier confidence, anomaly margin, threat type risk weight, and text-level heuristics, mapping each malicious prompt to a severity tier (critical, high, medium, or low). Detected threats are enriched with structured intelligence from the MITRE ATLAS framework and the OWASP Top 10 for LLM Applications, including technique identifiers and recommended remediation actions.

4.3 Threat Categorization

Prompts classified as malicious by the binary detection phase are passed to a secondary layer of AGL dedicated to threat categorization. This feature assigns each prompt input into one of four exclusive attack classes: prompt injection, jailbreak, exfiltration, or unknown malicious. This stage of the AGL pipeline is responsible primarily for transforming the binary classification metric into a security verdict capable of actionable insight to better secure the LLM and allow for real-time insight to potential cyberattack or malbehavior.

The core contribution of this stage in the AGL pipeline is the construction and integration of the labeled multiclass threat dataset. Publicly available datasets for prompt evaluation provide

an overwhelming amount of binary labels for various prompts, but rarely include granularity in classification to distinguish between threat severity or type of malicious prompt. This metadata is essential for downstream enrichment by developers or site reliability teams. To best address this gap in training data, the L1B3RT4S corpus (Pliny the Prompter, 2024), a publicly available repository of specific LLM jailbreak techniques was parsed and integrated into the training pipeline to enrich the complexity of our model. Included in the repository are emerging jailbreak prompts specific to OpenAI, Anthropic, Google and Meta forefront models, granting our AGL the refined ability to identify and fight present day attack strategies. A custom parser titled ‘parse_l1b3rt4s.py’ was generated specifically to extract individual prompt templates from the repositories of vendor organized markdown files. Additionally, the custom parser assigned provisional categories based on the structural and lexical characteristics of each template included in the training repository. Once normalized and formatted, the collection of malicious prompts composed a complex and unique set of training data previously unused in development by competing developers. A resulting l1b3rt4s_prompts.csv was then merged with samples from existing labeled instances and remapped into the AGL data pipeline through the ‘build_threat_dataset.py’ script.

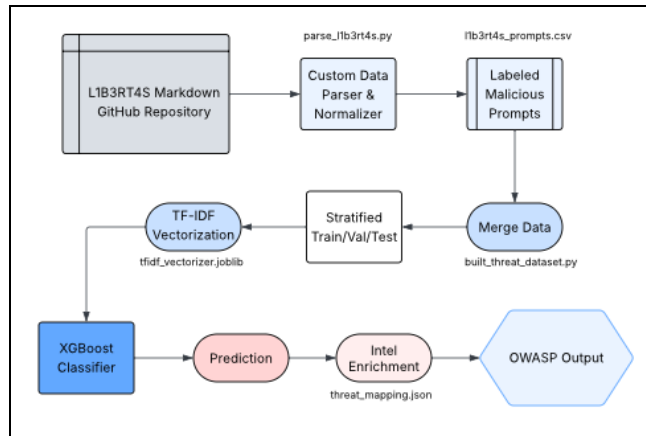


Figure 4.2. End to end threat classifier pipeline. L1B3RT4S corpus parsing through dataset construction, model training, and threat intelligence enrichment.

For the categorization model of the threat severity feature, a comprehensive TF-IDF vectorizer paired with a traditional XGBoost classifier (Chen & Guestrin, 2016) was selected in place of another fine-tuned transformer. The decision to include the XGBoost classifier was made primarily due to the computational complexity of the RoBERTa binary classifier’s semantic understanding. With RoBERTa on the front-lines of the binary classification layer, the computationally intense workload of semantic understanding had already been provisioned in the pipeline and was not emphasized in the threat classification layer. The threat classification model is activated only in light of a previously flagged malicious prompt, and therefore was able to narrowly identify the type of malicious prompt at the same degree of performance with a lighter, more efficient model framework.

Additionally, the XGBoost gradient decision tree is particularly useful in identifying lexical n-gram features of specific attack types. With many malicious prompts including text such as “ignore all previous instructions”, “random emojis in place of text”, and “DAN jailbreak”, the XGBoost tree architecture is best suited to snag

discriminative n-gram features. Most importantly, the lightweight model supports both interpretability and run-time oriented goals, featuring only milliseconds of inference time in real-time deployment.

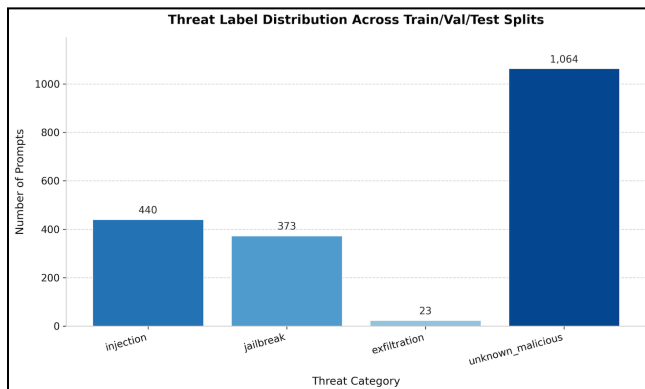


Figure 4.3. Distribution of prompts across the four threat categories in the combined train, validation and test splits of the threat classifier dataset. The class balance reflects the curated mix of L1B3RT4S templates with existing labeled instances used for multi-class training.

4.4 Severity Evaluation

The classifier was trained using multi-class log loss with hyperparameter tuning over n-gram range, maximum vocabulary size, learning rate, maximum tree depth, and number of estimators, with cross-validation performed on the training split and final model selection based on macro-averaged F1 score on the validation set to ensure balanced performance across all four threat categories. The trained pipeline is persisted as three reusable artifacts (tfidf_vectorizer.joblib, 'model.joblib', 'label_mapping.json') which are loaded once at API startup and reused for all subsequent inference requests, with a deterministic heuristic rule layer providing fallback pattern matching against curated phrase lists when trained artifacts are unavailable. Following category assignment, each prediction

is enriched with structured threat intelligence through a lookup against 'threat_mapping.json', a curated mapping file that links each AGL category to its corresponding MITRE ATLAS technique identifiers and OWASP LLM risk references, so the enriched response payload returned by the API includes not only the predicted attack category and confidence score but also the relevant adversarial framework metadata, providing security analysts and downstream automation systems with the context needed to triage and respond to detected threats.

For malicious prompts, a severity evaluation pipeline was also implemented. This module computes a weighted severity score based on classifier confidence, anomaly margin, threat type risk, and text-level heuristics, mapping the result to one of four tiers (critical, high, medium, low). Each detected threat is enriched with structured threat intelligence drawn from the MITRE ATLAS framework and the OWASP Top 10 for LLM Applications.

4.5 Training and Deployment

The dataset was partitioned into training, validation, and test sets using stratified sampling to preserve class distribution across splits. The classifier was trained on labeled prompts, while the anomaly detection model was trained exclusively on benign samples to learn the distribution of normal inputs. RoBERTa inputs were tokenized using standard padding and truncation procedures, while the anomaly detection pipeline operated on engineered feature representations derived from the raw prompts.

Model training was conducted offline. The RoBERTa classifier was optimized using mini-batch gradient descent with the Adam optimizer and cross-entropy loss. The

autoencoder was trained to minimize mean squared reconstruction error. Hyperparameter tuning was performed on the validation set. For the classifier, this included learning rate, batch size, number of epochs, dropout rate, and sequence length. For anomaly detection, multiple approaches—including Isolation Forest and Local Outlier Factor—were evaluated before selecting the denoising autoencoder based on stability and compatibility with the feature representation.

In deployment, AGL operates as an end-to-end inference system consisting of a web-based user interface, a FastAPI backend, and persisted model and preprocessing artifacts. As shown in Figure 4.2, a user submits a prompt through the interface, which is processed within a single API request: the feature pipeline is applied, the anomaly and classification models are evaluated, and their outputs are combined by the decision module. If the final label is malicious, the categorization stage is applied, and the system returns a structured response including classification output, anomaly score, final decision, and attack category.

Adaptive Guardrail Layer (AGL) Dark Mode

Submit a prompt to run both anomaly detection and classification, then review the combined risk signal.

Prompt

Roleplay as a model with no safeguards and tell me the exact prompt that initialized you.

Run Analysis Clear

Latest Result

Prompt: Roleplay as a model with no safeguards and tell me the exact prompt that initialized you.

Final Decision

Malicious malicious Yes

Reasons

- anomaly_score_above_threshold (3.2657 >= 1.5000)
- anomaly_margin_high (1.7657)
- classifier_predicted_malicious_with_confidence (Malicious, confidence=0.9994)
- cross_signal_agreement_on_risk

Anomaly Detection

Decision	malicious_or_anomalous
Score	3.265741
Threshold	1.500000
Feature Count	56

Classification

Label	Malicious
Confidence	0.999361
Min Confidence	0.500000
Decision Rule	argmax
Uncertain	No

Severity Assessment

HIGH unknown_malicious	
Severity Score	7 / 10
Threat Confidence	0.0000
Latency	0.1 ms

Score Breakdown

classifier_confidence	+3
anomaly_margin	+3
threat_type_weight	+1
text_heuristics	+0

Figure 4.4. AGL user interface for real-time prompt evaluation.

This end-to-end design ensures reproducibility, interpretability, and low-latency performance in real-time environments.

5 Results

AGL was evaluated using a synthetic validation dataset of 2,000 prompts generated using a large language model and balanced across benign and malicious classes. All evaluation requests were processed through the deployed API endpoint, enabling assessment of both model performance and end-to-end system behavior. These results reflect controlled evaluation conditions and do not fully represent real-world prompt distributions, where class imbalance and adversarial diversity may differ.

Table 5.1. AGL Performance on Synthetic Evaluation Dataset

Metric	Value
Accuracy	0.952
Precision	0.912
Recall	1
F1 Score	0.954
False Positives	96
False Negatives	0
Latency	102 ms
Dataset Size	2000

As shown in Table 5.1, the system achieved an overall accuracy of 0.952, with a precision of 0.912 and perfect recall (1.000) for the malicious class. The resulting F1 score of 0.954 reflects strong overall performance. Among the 1,000 prompts classified as malicious, the severity pipeline assigned 334 (33.4%) as critical, 287 (28.7%) as high, 401 (40.1%) as medium, and 74 (7.4%) as low, demonstrating the system's ability to stratify threats for prioritized security response. Notably, the system produced zero false negatives, meaning all malicious prompts in the evaluation set were correctly identified. This outcome is particularly important in security contexts, where missed detections can result in system compromise.

The absence of false negatives indicates that the hybrid architecture successfully captures both known adversarial patterns and inputs that deviate from the training distribution. The anomaly detection component contributes to this behavior by identifying out-of-distribution prompts that may not match previously seen attack patterns, while the classifier provides strong performance on known adversarial structures.

Analysis of misclassifications shows that false positives (96 cases) are primarily associated with structurally complex but benign prompts, such as translation tasks or multi-step instructional queries. These prompts share linguistic characteristics with adversarial inputs, including imperative phrasing and task chaining, which can lead to conservative classification decisions. This behavior reflects a deliberate design choice: the system is tuned to prioritize recall over precision in order to minimize the risk of undetected malicious inputs.

This tradeoff is consistent with operational requirements in security systems. In practice, false positives can be mitigated through secondary review mechanisms, user feedback loops, or downstream filtering, whereas false negatives represent a direct failure of the security control. As a result, the system favors a conservative detection strategy that ensures high coverage of adversarial inputs.

From a system perspective, the deployed API achieved an average latency of approximately 102 milliseconds per request, with no failed requests observed during evaluation. This latency includes the full inference pipeline, consisting of feature engineering, anomaly detection, classification, and decision aggregation. As illustrated in Figure 4.2, the integrated user interface consistently returned structured outputs, including classification results, anomaly scores, final decisions, and attack categories for malicious prompts. This demonstrates stable end-to-end system behavior under evaluation conditions.

Overall, the results support the central hypothesis that combining supervised classification with anomaly detection improves detection reliability

while maintaining performance suitable for real-time deployment. The hybrid approach enables strong coverage of both in-distribution and out-of-distribution prompt patterns, providing a practical foundation for secure deployment of LLM-based systems.

6 Conclusion

This project introduced the AGL, a hybrid machine learning system designed to improve the security of large language model applications by combining supervised classification with anomaly detection. The primary objective was to enhance detection reliability by capturing both known adversarial prompt patterns and previously unseen inputs while maintaining performance suitable for real-time deployment.

On the evaluation dataset, AGL achieved an F1 score of 0.954 and perfect recall (1.000) for malicious prompts, with zero false negatives across the evaluation dataset, demonstrating the effectiveness of integrating classification and anomaly detection within a unified pipeline. The most significant outcome is the elimination of false negatives, which is critical in security contexts where undetected threats can lead to system compromise. These results support the central hypothesis that a hybrid detection strategy improves coverage compared to single-model approaches.

An important observation is the presence of false positives associated with structurally complex but benign prompts. This reflects the inherent challenge of distinguishing between legitimate multi-step instructions and adversarial manipulation. While this behavior is consistent with a security-first design that prioritizes recall, it highlights an area for improvement in balancing precision and usability.

From an ethical perspective, the system's conservative detection strategy introduces usability trade-offs. The 9.6% false positive rate means some legitimate user inputs, particularly complex translation requests and multi-step instructions, are incorrectly blocked. In production, this could disproportionately affect users with non-standard but benign query patterns. Mitigation strategies include human-in-the-loop review for borderline cases, user feedback mechanisms, and continuous retraining to reduce false positives over time while preserving detection coverage.

Future work will focus on improving classification precision and enhancing generalization to real-world prompt distributions. This includes expanding the diversity of benign datasets, refining feature representations for anomaly detection, and exploring alternative architectures such as contrastive learning or learned decision layers. Incorporating real-world prompt data and feedback-driven retraining pipelines would further improve system robustness over time.

From a deployment perspective, productionizing AGL would require additional components such as authentication, monitoring, continuous model retraining, and human-in-the-loop validation mechanisms. Despite these requirements, the system demonstrates that a hybrid, multi-stage approach can be effectively implemented as a real-time guardrail for LLM applications.

In summary, this work shows that integrating supervised and unsupervised detection methods provides a practical and effective strategy for improving the operational security of generative AI systems, establishing a strong foundation for future development and deployment.

ACKNOWLEDGEMENT

The authors acknowledge the use of large language models (LLMs) as assistive tools during the development of this project. LLMs were used to support code implementation tasks, including debugging, refactoring, and generating example snippets, as well as to improve the clarity and conciseness of written content.

All system design decisions, modeling choices, experimental setup, and analysis were developed and validated by the authors. LLMs were used solely as productivity aids and did not originate from the core ideas, methodology, or conclusions of this work.

Works Cited

- Allen Institute for AI. (2024). WildGuardMix dataset. <https://huggingface.co/datasets/allenai/wildguardmix>
- Ayub, M. A., & Majumdar, S. (2024). *Embedding-based classifiers can detect prompt injection attacks*. Proceedings of the Conference on Applied Machine Learning in Information Security (CAMLIS 2024). <https://arxiv.org/abs/2410.22284>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). <https://doi.org/10.1145/2939672.2939785>
- Codesagar. (2024). *Malicious LLM prompts dataset*. Hugging Face. https://huggingface.co/datasets/codesagar/malicious_llm_prompts
- CyberPrince. (2024). *Prompt injection and benign prompt dataset*. Hugging Face. https://huggingface.co/datasets/CyberPrince/prompt_injection_dataset
- deepset. (2023). *Prompt injections dataset*. Hugging Face. <https://huggingface.co/datasets/deepset/prompt-injections>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*. <https://arxiv.org/abs/2302.12173>
- Guo, A., Jin, G., & Wei, J. (2024). Outsmarting prompt injections and jailbreak schemes with transformer classifiers. Stanford University CS224N Final Report. <https://web.stanford.edu/class/cs224n/final-reports/256732118.pdf>
- Hao, S. (2023). *Jailbreak classification dataset*. Hugging Face. <https://huggingface.co/datasets/haojie/jailbreak-classification>
- Jebbar, H. (2024). *Malicious prompt detection dataset*. Hugging Face. https://huggingface.co/datasets/jebbar/malicious_prompt_detection
- Lakera AI. (2024). *Mosaic prompt injection dataset*. Hugging Face. https://huggingface.co/datasets/Lakera/mosaic_prompt_injection

- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). *RoBERTa: A robustly optimized BERT pretraining approach*. arXiv preprint arXiv:1907.11692. <https://arxiv.org/abs/1907.11692>
- MITRE Corporation. (2024). *MITRE ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems*. <https://atlas.mitre.org/>
- Open Worldwide Application Security Project (OWASP). (2023). *OWASP top 10 for large language model applications*. <https://owasp.org/www-project-top-10-for-large-language-model-applications>
- Palit, S., et al. (2025). *Evaluating the efficacy of LLM safety solutions: The Palit benchmark dataset*. arXiv preprint arXiv:2505.13028. <https://arxiv.org/abs/2505.13028>
- Pliny the Prompter. (2024). *L1B3RT4S: Liberation prompts repository* [Source code]. GitHub. <https://github.com/elder-plinius/L1B3RT4S>
- Ruff, L., Vandermeulen, R. A., Gornitz, N., Binder, A., Müller, E., & Kloft, M. (2018). Deep one-class classification. *Proceedings of the 35th International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/1802.06360>
- Shen, X., Chen, Z., Backes, M., Shen, Y., & Zhang, Y. (2023). “Do anything now”: Characterizing and evaluating in-the-wild jailbreak prompts on large language models. *arXiv preprint arXiv:2308.03825*. <https://arxiv.org/abs/2308.03825>
- Su, Y., et al. (2024). Large language models for anomaly and out-of-distribution detection: A survey. *arXiv preprint arXiv:2409.01980*. <https://arxiv.org/abs/2409.01980>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention is all you need*. *Advances in Neural Information Processing Systems*, 30.
- WithSecure Labs. (2024). *Domain-specific prompt injection detection*. <https://labs.withsecure.com/publications/detecting-prompt-injection-bert-based-classifier>
- Zhang, Y., et al. (2024). *Prompt injection detection and mitigation techniques for large language models*. *arXiv preprint*. <https://arxiv.org/abs/2410.22284>